GUÍA DE TRABAJO / SPRINT 1 · ACTUALIZADA

Desarrollo y gestión de ramas

Proyecto XFood · Distribución del equipo y procedimiento de integración

Acuerdo: cada integrante hace push a su rama feature existente y abre un pull request hacia develop. Los pull requests de las historias no tendrán como destino main.

1. Contexto y punto de partida

XFood cuenta con una API ASP.NET Core organizada como monolito modular y una base relacional SQL Server. Para este sprint se prioriza la aplicación web con React + Next.js; la aplicación móvil se desarrollará posteriormente.

Según el informe del equipo, las migraciones iniciales están preparadas y aplicadas en un entorno local. Cada integrante obtiene la solución, configura su propia conexión y aplica las migraciones en su base local siguiendo el README. Los casos de uso y endpoints se desarrollan durante las historias.

Función de las ramas

Rama	Uso acordado
main	Rama principal; no recibe pull requests de estas historias.
develop	Rama común de integración; recibe los pull requests revisados.
feature/...	Ramas individuales ya creadas; cada persona realiza sus commits y push en la suya.

Las seis ramas de la siguiente sección ya existen. Las claves de Jira se pueden registrar en commits o PR cuando se verifiquen; no es necesario cambiar el nombre de las ramas para seguir este flujo.

2. Historias, responsables y ramas

Cada persona trabaja en la rama que ya le corresponde. Las parejas frontend/backend acuerdan las rutas de la API, los datos de entrada y salida, los errores y los permisos.

Historia	Backend / rama existente	Frontend / rama existente
HU-01 · Publicación de excedentes	Carlos `feature/hu-01-backend`	Juan Eliud `feature/hu-01-frontend`
HU-02 · Búsqueda y filtrado	Diego Alonso `feature/hu-02-backend`	Diana `feature/hu-02-frontend`
HU-04 · Registro e inicio de sesión	Wilmer `feature/hu-04-backend`	Michael `feature/hu-04-frontend`

Base web compartida: Juan Eliud

Juan Eliud es responsable de preparar la base común de Next.js. Usará su rama existente `feature/hu-01-frontend`; no necesita crear otra rama. La base debe incluir estructura inicial, configuración de la URL de la API por entorno y una guía de ejecución sin credenciales.

Para que Diana y Michael puedan comenzar pronto, Juan Eliud puede publicar primero solo la base compartida en su rama y abrir un pull request hacia develop. El equipo revisa e integra esa parte; después, cada integrante actualiza su rama con develop. Juan continúa HU-01 en su misma rama y abre otro PR cuando corresponda. Si GitHub ofrece borrar la rama al fusionar el primer PR, debe conservarse para continuar.

> La aplicación de las migraciones crea el esquema de datos; cada equipo debe desarrollar y probar por separado sus casos de uso, rutas y pantallas.

3. Orden recomendado de trabajo

01 · Preparar el entorno

Cada integrante descarga o actualiza el proyecto, configura SQL Server local y aplica las migraciones existentes según el README. Verificar el nombre exacto de develop y que las ramas feature estén disponibles en el remoto.

02 · Acordar contratos

Wilmer y Michael coordinan el acceso; Carlos y Juan Eliud acuerdan publicaciones y categorías; Diego Alonso y Diana acuerdan búsqueda y ubicación. Definir primero el contrato de API permite trabajar en paralelo.

03 · Integrar la base web

Juan Eliud desarrolla la base común en feature/hu-01-frontend y la envía por PR a develop. Diana y Michael incorporan develop a sus ramas después de la integración.

04 · Desarrollar las historias

Backend y frontend avanzan en sus propias ramas. HU-01 depende de establecimiento, permisos y categorías; HU-02 necesita ofertas y ubicación. Los datos simulados son temporales durante el desarrollo.

05 · Revisar e integrar

Cada integrante hace push a su feature y abre un PR hacia develop. Un compañero revisa cambios y pruebas; después de integrar, los demás actualizan sus ramas y comprueban la integración.

06 · Demostrar el incremento

El equipo verifica en develop los recorridos con la API real y el PO revisa los criterios acordados. Se registra la evidencia en Jira.

4. Trabajar con una rama existente

Desde la carpeta del repositorio, revisa primero tu estado y actualiza las referencias. Los ejemplos suponen que origin es el remoto correcto y que la rama feature ya existe allí.

```
git status
git fetch origin
git branch -a
```

Si tienes cambios sin guardar o conflictos, no los descartes: revísalos con el equipo antes de cambiar de rama.

Cambiar a tu rama

Ejemplo para Carlos. Si la rama ya existe en tu computadora:

```
git switch feature/hu-01-backend
```

Si existe en GitHub como origin/feature/hu-01-backend pero aún no está localmente:

```
git switch --track origin/feature/hu-01-backend
```

El segundo comando crea la copia local de seguimiento de la rama existente; no crea una rama nueva en GitHub. Cada integrante sustituye el nombre por el suyo.

Incorporar cambios de develop

Con tu trabajo guardado o comprometido y estando en tu rama feature:

```
git fetch origin
git merge origin/develop
```

Si aparecen conflictos, resuélvelos conservando los cambios necesarios y vuelve a probar. La rama de integración no se sobrescribe con este comando.

Guardar y subir tus cambios

```
git status
git add <archivos-revisados>
git diff --cached
git commit -m "Implement excess listing operations"
git push -u origin feature/hu-01-backend
```

Sustituye las rutas, el mensaje y la rama por los correspondientes a tu trabajo. No incluyas credenciales ni archivos locales ignorados.

5. Pruebas de las implementaciones

Cada integrante es responsable de probar los cambios de su rama antes de solicitar revisión. Las pruebas deben comprobar el comportamiento y las reglas de negocio relevantes para la implementación.

Pruebas unitarias del backend

Utilizar xUnit, que ya está presente en el proyecto, para probar reglas y casos de uso de forma aislada. Cubrir situaciones correctas y errores importantes, sin depender necesariamente de una base de datos real.

Historia	Ejemplos de comportamiento a probar
HU-04 · Acceso	Credenciales inválidas; validación del registro; restricciones por rol según el alcance implementado.
HU-01 · Publicaciones	Cantidad positiva; precio de venta válido; rechazo de modificación sin autorización; modalidad y estado permitidos.
HU-02 · Búsqueda	Validación de ubicación y radio; resultados y reglas de filtrado conforme al contrato implementado.

Pruebas de integración y frontend

Cuando el cambio involucre API, persistencia, permisos o transacciones, comprobar también que los componentes funcionen juntos. Para concurrencia o restricciones SQL Server, usar una base de pruebas aislada y nunca depender de los datos personales del equipo. En frontend, probar formularios, validaciones, carga y errores; verificar el recorrido contra la API real antes de darlo por terminado.

Ejecución y evidencia

Desde la carpeta que contiene XFood.slnx, ejecutar el comando siguiente para las pruebas .NET de la solución. Seguir el README y los scripts reales del frontend para sus pruebas; no asumir que ya existe un comando de pruebas web.

```
dotnet test XFood.slnx
```

Si una prueba requiere configuración adicional o no puede ejecutarse, informar el motivo. Incluir en el pull request qué se probó, el resultado y las verificaciones pendientes. Los casos de prueba evolucionan con cada módulo y sus requisitos.

6. Pull request y seguimiento

1. Al completar una parte revisable, crea un pull request desde tu **rama feature existente** hacia **develop**. Verifica origen y destino antes de enviarlo.

2. Describe los cambios, cómo probarlos y las dependencias. Incluye la clave real de Jira si corresponde; no inventes claves.

3. Solicita revisión a otro integrante, ejecuta las pruebas pertinentes y atiende los comentarios. Resuelve conflictos sin borrar trabajo ajeno.

4. Integra en develop después de la revisión acordada. Actualiza Jira con el estado y la evidencia del recorrido.

Lista de comprobación

• Mi rama corresponde a la asignación del equipo.

• Agregué o actualicé pruebas pertinentes, el código compila y documento los resultados y lo que no pude verificar.

• La interfaz utiliza el contrato acordado y muestra carga, errores y estados vacíos.

• He revisado permisos, secretos y migraciones cuando mi cambio las requiere.

• El destino del pull request es develop.

Base web: Juan Eliud. Rama: feature/hu-01-frontend. Las seis ramas ya existen y no es necesario crear otras para comenzar.

Informe actualizado con los responsables y ramas indicados por el equipo.